

BUỔI 11: Kiểu tệp tin (file) – Tệp tin văn bản

I. Lý thuyết

1. Định nghĩa

Tệp tin (file) hay tệp dữ liệu là một tập hợp các dữ liệu có liên quan với nhau và có cùng một kiểu được nhóm lại với nhau thành một dãy. Chúng thường được chứa trong một thiết bị nhớ ngoài của máy tính (đĩa mềm, đĩa cứng...) dưới một cái tên nào đó.

Tệp là một kiểu dữ liệu có cấu trúc. Định nghĩa tệp có phần nào giống mảng ở chỗ chúng đều là tập hợp của các phần tử dữ liệu cùng kiểu, song mảng thường có số phần tử cố định, số phần tử của tệp không được xác định trong định nghĩa.

Một tệp tin dù được xây dựng bằng cách nào đi nữa cũng chỉ đơn giản là một dãy các byte ghi trên đĩa (có giá trị từ 0 đến 255). Số byte của dãy chính là độ dài của tệp.

Có hai kiểu nhập xuất dữ liệu lên tệp :

- Nhập xuất nhị phân
Nhập xuất văn bản.

2. Làm việc với file văn bản

+ Mở file:

Để mở một file làm việc ta sử dụng khai báo sau:

```
FILE *fopen( const char * ten_file, const char * che_do );
```

Hoặc:

```
FILE *<con_tro_file>
```

```
Con_tro_file = fopen( const char * ten_file, const char * che_do );
```

Trong đó:

- Ten_file: là một chuỗi ký tự đại diện cho tên của file đã có hoặc tạo mới. Nếu không chỉ ra đường dẫn chứa file thì chương trình sẽ ngầm hiểu file này ở trong thư mục chứa mã nguồn của project.
- Che_do: là các chế độ xử lý file, bao gồm :

Chế độ	Giải thích
r	Mở các file đã tồn tại với mục đích đọc
w	Mở các file đã tồn tại với mục đích ghi. Nếu các file này chưa tồn tại thì file mới sẽ được tạo ra. Ở đây, chương trình bắt đầu ghi nội dung từ phần mở đầu của file.
a	Mở file văn bản cho việc ghi ở chế độ ghi tiếp theo vào cuối, nếu file chưa tồn tại thì file mới được tạo ra. Ở đây, chương trình ghi nội dung với phần cuối file đã tồn tại.
r+	Mở các file đã tồn tại với mục đích đọc và ghi
w+	Mở các file đã tồn tại với mục đích đọc và ghi. Nếu các file này đã có thì nó sẽ xóa sạch các dữ liệu và nếu file chưa tồn tại thì file mới sẽ được tạo ra.
a+	Mở file văn bản đã tồn tại với mục đích đọc và ghi. Nó tạo file mới nếu không tồn tại. Việc đọc file sẽ bắt đầu đọc từ đầu nhưng ghi file sẽ chỉ ghi vào cuối file.

Ví dụ: Khai báo một biến con trỏ file có tên là fp, tên file con trỏ trỏ tới là “text.txt”. Mở file để đọc dữ liệu.

```
FILE *fopen("text.txt", "r") ;
```

Hoặc

```
FILE *fp ;  
fp = fopen("text.txt", "r");
```

+Ghi File:

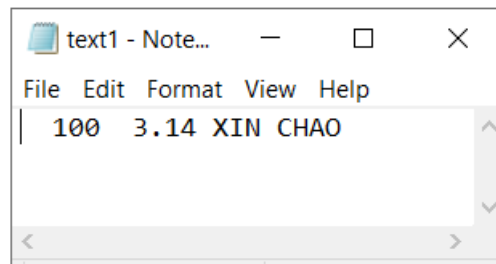
Để ghi nội dung vào file, ta sử dụng hàm `fprintf()` với cú pháp như sau:

```
fprintf(FILE *f, const_char *format, ...)
```

Ví dụ 1: Ghi 1 số nguyên, 1 số thực và 1 chuỗi ký tự vào file `text1.txt`

```
int main()  
{  
    FILE *fp;  
    fp= fopen("text1.txt", "w");  
    char S[]="XIN CHAO";  
    fprintf(fp, "%5d %5.2f %s", 100, 3.14, S);  
    fclose(fp);  
  
    return 0;  
}
```

Kết quả, chương trình tạo 1 file có tên là `text1.txt` với nội dung như sau:



Ví dụ 2: Ghi vào file `text2.txt` các số nguyên tố nhỏ hơn `n`, mỗi dòng tối đa 10 số

```
//Hàm kiểm tra nguyên tố  
int NT(int a)  
{  
    int i;  
    if (a<2) return 0;  
    for (i=2;i<=sqrt(a);i++)  
        if (a%i==0) return 0;  
    return 1;  
}  
  
//Hàm ghi số nguyên tố vào file  
void WriteNT(FILE *fp, char NameF[20],int n)  
{  
    fp= fopen(NameF,"w");//Tham số w trong fopen cho phép ghi  
    đè dữ liệu lên file đã có.  
    int i,dem =0;  
    fprintf(fp,"Cac so nguyen to nho hon %d la:\n",n);  
    for (i=2;i<=n;i++)  
        if (NT(i)==1)
```

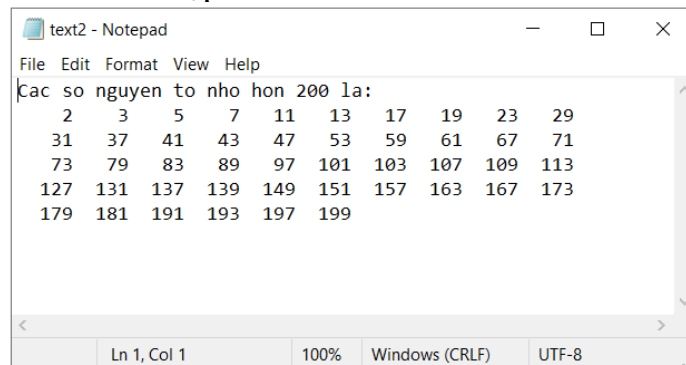
```

    {
        fprintf(fp, "%5d", i);
        dem++;
        if (dem%10==0)
            fprintf(fp, "\n");
    }
    fclose(fp);
}

int main()
{
    FILE *fp;
    int n;
    printf("Nhap n:"); scanf("%d", &n);
    WriteNT(fp, "text2.txt", n);
    return 0;
}

```

Kết quả của file text2.txt sau khi nhập n=200 :

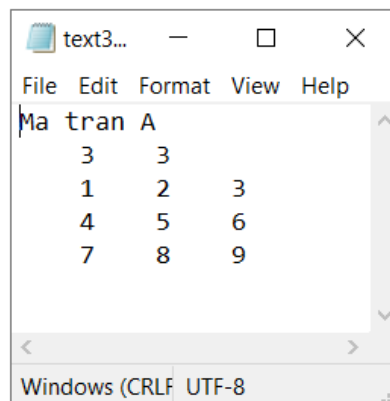


Ví dụ 3 : Ghi vào file text3.txt ma trận A có n hàng và m cột theo yêu cầu sau :

Dòng đầu tiên của file ghi số hàng, số cột

Các dòng tiếp theo ghi phần tử của ma trận

Cụ thể : đây là file chứa ma trận A có 3 hàng, 3 cột



Chương trình :

//Ghi ma trận vào file với yêu cầu: dòng 1 ghi số hàng, số cột. Từ dòng 2 ghi ma trận

```

void WriteMT(FILE *fp, char NameF[20], char ArrName)
{
    int n,m,i,j;
    int A[100][100];

```

```

        fp=fopen(NameF,"a");//Tham số a trong hàm fopen cho phép ghi
tiếp dữ liệu vào cuối file
        fprintf(fp,"Ma tran %c \n",ArrName);
        printf("nhap so hang ma tran:");
        scanf("%d",&n);
        printf("Nhap so cot ma tran:");
        scanf("%d",&m);
        //Ghi số hàng số cột vào ma trận
        fprintf(fp,"%5d%5d\n",n,m);
        for (i=0;i<n;i++)
        {
            for (j=0;j<m;j++)
            {
                printf("%c[%d,%d]=",ArrName,i,j);
                scanf("%d",&A[i][j]);
                //Ghi từng phần tử của ma trận vào file
                fprintf(fp,"%5d",A[i][j]);
            }
            fprintf(fp,"\n");
        }
        fclose(fp);
    }

int main()
{
    FILE *fp;
    //Lời gọi hàm WriteMT với ba tham số, fp là con trỏ file,
    text3.txt là tên file, A là tên ma trận
    WriteMT(fp,"text3.txt",'A');
    return 0;
}

```

Chú ý: Chạy thử chương trình với ma trận B (thay đổi tham số A trong lời gọi hàm thành B), mở file text3.txt để quan sát và đánh giá.

+ Đọc File:

Để đọc nội dung từ file ta sử dụng ba hàm : fscanf(), fgets(), fgetc().

* **Hàm fscanf()** : dùng để đọc dữ liệu từ file, hàm này cho phép đọc 1 chuỗi không có dấu cách. Nếu chuỗi có dấu cách, hàm chỉ đọc được phần dữ liệu trước dấu cách đầu tiên. Cú pháp :

```
int fscanf(FILE *f, const char * format, ...)
```

Ví dụ 1: đọc dữ liệu trong file text1.txt và in kết quả ra màn hình

```

int main()
{
    FILE *fp;
    int x;
    float y;
    char s[100];
    fp=fopen("text1.txt","r");
    if (fp==NULL)
        printf("Cannot open File");
}

```

```

else
{
    fscanf(fp, "%d%f%s", &x, &y, &s);
    printf("\nKet qua doc file:");
    printf("\nx=%d", x);
    printf("\ny=%5.2f", y);
    printf("\nS=%s", s);
    fclose(fp);
}
return 0;
}

```

Nhận xét: Do chuỗi trong file text.txt có chứa dấu cách nên khi đọc bằng hàm fscanf() sẽ bị mất thông tin trong chuỗi. Ta cần sử dụng fgets() để đọc chuỗi như sau:

```

int main()
{
    FILE *fp;
    int x;
    float y;
    char s[100];
    fp=fopen("text1.txt", "r");
    if (fp==NULL)
        printf("Cannot open File");
    else
    {
        fscanf(fp, "%d%f", &x, &y);
        fgets(s, 100, fp); //Hàm đọc chuỗi từ file
        printf("\nKet qua doc file:");
        printf("\nx=%d", x);
        printf("\ny=%5.2f", y);
        printf("\nS=%s", s);
        fclose(fp);
    }
    return 0;
}

```

Ví dụ 2 : Viết chương trình đọc mảng nguyên từ file **input.txt** và ghi các số nguyên tố trong mảng vào file **output.txt**. Biết rằng, file **input.txt** có cấu trúc như sau : dòng đầu tiên ghi số phần tử của mảng. Dòng tiếp theo ghi các phần tử mảng. File **output.txt** ghi các số nguyên tố của mảng, mỗi dòng có tối đa 10 số. Một ví dụ về hai file.

input.txt	Ouput.txt
30	
2 5 4 10 6 20 45 9 11 17 62 13 15 17 31 40	2 5 11 17 13 17 31 7 13 23
47 52 16 7 13 30 23 18 19 21 40 50 51 60	19 51

Hướng dẫn:

```

void ReadFile2(FILE *f1, FILE *f2)
{
    int n,i,x,d=0;

    f1=fopen("input.txt","r");//Mở file input để đọc dữ liệu
    if (f1==NULL)
        printf("Cannot open File");
    else
    {
        f2=fopen("output.txt","w");//Mở file output để ghi dữ liệu
        fscanf(f1,"%d",&n);
        for (i=0;i<n;i++)
        {
            fscanf(f1,"%d",&x);
            if (NT(x)==1)
            {
                fprintf(f2,"%5d",x);
                d++;
                if (d%10==0) fprintf(f2,"\n");
            }

        }

        fclose(f1);
        fclose(f2);
    }
}

```

Ví dụ 3: Viết chương trình đọc ma trận từ file **input1.txt** và sắp xếp ma trận theo chiều tăng dần của hàng và ghi kết quả vào file **output1.txt**. Biết rằng, file **input1.txt** có cấu trúc như sau : dòng đầu tiên ghi số hàng, số cột của ma trận. Các dòng tiếp theo ghi các phần tử ma trận. File **output1.txt** có cấu trúc như sau: dòng đầu ghi số hàng, số cột của ma trận. Các dòng tiếp theo ghi ma trận đã được sắp xếp theo chiều tăng dần của hàng. Ví dụ về hai file.

Input1.txt	Ouput1.txt
3 4	3 4
4 5 2 1	1 2 4 5
7 9 1 3	1 3 7 9
8 0 6 4	0 4 6 8

Hướng dẫn :

```

#include<conio.h>
#include<stdio.h>
void ReadFile3(FILE *f1, FILE *f2)
{
    int n,m,i,j,k,tg;
    f1=fopen("input1.txt","r");
    if (f1==NULL)
        printf("File not found!");
    else
    {
        //Đọc file input3.txt
        fscanf(f1,"%d%d",&n,&m);
        int A[n][m];
        for (i=0;i<n;i++)
        {
            for (j=0;j<m;j++)
                fscanf(f1,"%d",&A[i][j]);
        }
    }
}

```

```

//Sắp xếp mảng theo chiều tăng của hàng
for (i=0;i<n;i++)
{
    for (j=0;j<m;j++)
        for (k=j+1;k<m;k++)
            if (A[i][j]>A[i][k])
            {
                tg=A[i][j];
                A[i][j]=A[i][k];
                A[i][k]=tg;
            }
}
//Ghi mảng vào file output2.txt
f2=fopen("output2.txt","w");
fprintf(f2,"%5d %5d\n",n,m);
for (i=0;i<n;i++)
{
    for (j=0;j<m;j++)
        fprintf(f2,"%4d",A[i][j]);
    fprintf(f2,"\n");
}
fclose(f1);
fclose(f2);
}

}
int main()
{
    FILE *f1,*f2;
    ReadFile3(f1,f2);
    return 0;
}

```

II. Bài tập

Câu 33: Viết chương trình tính tổng hai ma trận: $C_{n \times m} = A_{n \times m} + B_{n \times m}$. Các ma trận A, B được nhập vào từ bàn phím. Ghi ba ma trận vào tệp CONG_MT.C (sử dụng kiểu nhập xuất văn bản), rồi mở tệp này ra để xem kết quả.

Câu 34: Viết chương trình tính tích hai ma trận: $C_{n \times m} = A_{n \times p} \times B_{p \times m}$. Trong đó dữ liệu về: n, p, m và hai ma trận A, B được ghi trên tệp TICH_MT.C. Ghi bổ sung ma trận tích tính được vào cuối tệp này (sử dụng kiểu nhập xuất văn bản), rồi mở nó ra để xem kết quả.